

Architectural Blueprint and Data Specification for an Ultra-Lightweight Carbon Footprint Tracking Platform

The development of a highly granular individual carbon footprint tracking web application within a strict repository size limit of under ten megabytes represents a significant challenge in modern software engineering. Contemporary web development paradigms frequently rely on monolithic dependency trees, locally hosted geographical datasets, and heavy client-side visualization libraries. Such architectures inevitably bloat repository size, increase client-side payload, and paradoxically generate a larger digital carbon footprint through excessive data transmission and computational overhead. To satisfy the stringent ten-megabyte constraint while delivering a robust, scientifically accurate environmental data platform, the architecture must abandon traditional stateful data hosting in favor of a stateless calculation engine. This approach necessitates the utilization of edge-optimized Application Programming Interfaces (APIs) for remote data aggregation, strict dependency austerity, and the implementation of native Document Object Model (DOM) rendering methodologies, such as cascading style sheets (CSS) only graphics and native Scalable Vector Graphics (SVG).

This comprehensive technical dossier provides the exact mathematical specifications, algorithmic formulas, data integration blueprints, and psychological frameworks required to construct this ultra-lightweight, highly performant carbon tracking solution. The methodology integrates validated environmental data science with advanced web architecture, ensuring strict compliance with internationally recognized greenhouse gas (GHG) reporting standards while maintaining an absolute minimal source code footprint.

1. The Calculation Engine (Formulas & Coefficients)

A stateless calculation engine forms the functional core of the application. By hardcoding universally recognized emission factors directly into a compiled utility module, the application circumvents the need for complex database queries or the inclusion of massive local structured data files, thereby directly satisfying the repository size constraint. The emission factors utilized within this engine are derived from the most current and authoritative environmental reporting guidelines, predominantly the Department for Environment, Food & Rural Affairs (DEFRA) 2024/2025 greenhouse gas conversion factors¹, the United States Environmental Protection Agency (EPA) Greenhouse Gas Emission Factors Hub⁵, and rigorously peer-reviewed epidemiological and environmental literature.⁷

Calculations across this platform are uniformly expressed in kilograms of Carbon Dioxide equivalent ($\text{kg CO}_2\text{e}$). This metric is essential for standardizing the global warming potential (GWP) of distinct greenhouse gases. While carbon dioxide (CO_2) is the most prevalent

anthropogenic greenhouse gas, others such as methane (CH_4) and nitrous oxide (N_2O) possess significantly higher heat-trapping capacities. Methane, for instance, has a GWP 28 times greater than carbon dioxide over a 100-year timescale, while nitrous oxide is 265 times more potent according to the Intergovernmental Panel on Climate Change (IPCC) Fifth Assessment Report (AR5) standard.⁶ The $\text{kg CO}_2\text{e}$ metric unifies these disparities into a single, comprehensible value.

Transport Emissions Specifications

Transport calculations are fundamentally deterministic, relying on a mathematical synthesis of the chosen mode of transport, the spatial distance traversed, and the established energy efficiency of the specific vehicle class. The primary formula governing transport emissions is defined as:

$$E_{transport} = D \times EF_{mode}$$

Where $E_{transport}$ represents the total generated emissions in $\text{kg CO}_2\text{e}$, D signifies the distance traveled in kilometers (km), and EF_{mode} represents the specific emission factor for the transport mode expressed in $\text{kg CO}_2\text{e per km}$.

The 2024 and 2025 DEFRA GHG Conversion Factors for Company Reporting establish the baseline values for passenger vehicles. These factors are critically important because they incorporate both direct tailpipe emissions, known as Tank-to-Wheel (TTW), and upstream lifecycle emissions associated with the extraction, refinement, and transportation of the fuel, known as Well-to-Tank (WTT).¹ Incorporating WTT factors ensures a holistic representation of the user's true environmental impact. Furthermore, recent methodological updates have adjusted the utility factors for plug-in hybrid electric vehicles (PHEVs) to better align with real-world operational data, resulting in a slight increase in direct fuel emission estimations for these vehicle types.¹¹

Transport Mode	Emission Factor (EFmode)	Unit of Measurement	Source Context
Average Gasoline (Petrol) Car	0.21 $\text{kg CO}_2\text{e}$	per vehicle-km	Incorporates TTW and WTT emissions for standard combustion engines. ¹⁰

Average Diesel Car	0.21 kg CO ₂ €	per vehicle-km	Aligning closely with gasoline averages due to modern efficiency standards. ¹⁰
Battery Electric Vehicle (BEV)	0.06 kg CO ₂ €	per vehicle-km	Accounts for grid generation intensity and transmission losses; zero tailpipe emissions. ¹⁰
Local Transit Bus	0.13 kg CO ₂ €	per passenger-km	Adjusted for post-COVID occupancy rates derived from transit statistics. ¹⁰
Subway / Light Rail / Metro	0.02 kg CO ₂ €	per passenger-km	High-efficiency electrified mass transit systems yield the lowest per-capita footprint. ¹⁰

While Battery Electric Vehicles (BEVs) produce zero direct tailpipe emissions, the carbon intensity of the electricity grid from which they draw power dictates their lifecycle footprint. For a standardized, offline approximation, the general emission factor of

0.06 kg CO₂e/km is utilized.¹⁰ However, for a dynamic calculation that leverages real-time API integrations, the application must compute

$E_{bev} = D \times E_{efficiency} \times CI_{grid}$, where $E_{efficiency}$ represents the vehicle's inherent consumption rate (averaging **0.15 kWh/km** for standard sedans) and CI_{grid} represents the localized grid carbon intensity at the time of charging.

Home Energy Specifications

Home energy calculations translate volumetric or power-based consumption metrics into greenhouse gas emissions. This domain primarily encompasses natural gas utilized for domestic heating and grid electricity consumed by household appliances and lighting.

Natural gas is typically metered and billed in either cubic meters (**m³**) or kilowatt-hours (**kWh**). The conversion from a volumetric measurement to an equivalent energy measurement

requires factoring in the gross calorific value of the gas, which can vary slightly based on regional chemical composition. Based on extensive DEFRA and EPA modeling, the direct combustion of natural gas yields highly consistent emission factors. The EPA calculates natural gas emissions at approximately **53.06 kg CO₂** per million British thermal units (mmBtu).⁶ When standardized into metric volumetric and power units suitable for a consumer-facing application, the factors are distilled as follows:

Energy Source	Emission Factor	Unit	Derivation Context
Natural Gas	2.50 kg CO₂e	per cubic meter (m ³)	Standardized conversion for volumetric domestic utility billing. ¹²
Natural Gas	0.18290 to 0.203 kg	per kilowatt-hour (kWh)	Gross calorific value basis; varies slightly by regional gas mix. ¹²
Liquid Petroleum Gas (LPG)	0.21450 to 0.230 kg	per kilowatt-hour (kWh)	Utilized in rural or off-grid residential heating scenarios. ¹²

Electricity emission factors represent the most geographically volatile component of the carbon tracking algorithm. The carbon intensity of an electrical grid is inextricably linked to its generation mix—the proportion of coal, natural gas, nuclear, hydroelectric, and variable renewable energy sources supplying the grid at any given moment. For static baseline calculations, which are necessary when external API calls fail or when offline functionality is required, the application must default to national or regional averages. The calculation multiplies the total consumed energy by the grid's carbon intensity:

$$E_{elec} = C_{kWh} \times EF_{grid}$$

To contextualize this volatility, the United Kingdom's average grid intensity is approximately **0.207 kg CO₂e/kWh**.¹⁵ In stark contrast, a heavily fossil-fuel-dependent grid, such as that of India, exhibits an average carbon intensity ranging from **0.710** to **0.82 kg CO₂e/kWh**, primarily due to the fact that coal power generates over 72% of the

nation's electricity and accounts for roughly 95.7% of its grid emissions.¹⁶ Consequently, operating identical electrical appliances in different geographic regions yields vastly different carbon footprints, necessitating the localized API integrations detailed in subsequent sections.

Dietary Specifications

Dietary footprint calculations present a unique challenge, as they cannot be metered in the traditional sense. Instead, they are derived from extensive epidemiological and life-cycle assessment studies. The foundational framework for this application relies upon the exhaustive study conducted by Scarborough et al. (2014), which analyzed the dietary greenhouse gas emissions of distinct, self-selected demographic groups in the United Kingdom.⁷

The Scarborough study normalized emissions against a standard 2,000 kilocalorie daily intake to isolate the environmental impact of the food type from the total volume of food consumed.¹⁹ The research conclusively demonstrated that dietary greenhouse gas emissions in high meat-eaters are approximately twice as high as those in vegans.⁷ The carbon footprint of meat, particularly ruminant livestock such as beef and lamb, includes enteric fermentation (methane release), intensive land use, and the significant water footprint required for animal agriculture.¹⁹ To implement this data within a daily tracking web application, the age-and-sex-adjusted mean daily values from the Scarborough study are utilized as baseline constants.⁷ To allow users to log individual meals rather than entire days, these daily values are mathematically divided into per-meal coefficients, assuming a standard model of three meals per day.

Dietary Classification	Daily Emission Factor	Per-Meal Coefficient	Study Definition (Scarborough et al.)
High-Meat Diet	7.19 kg CO ₂ €	2.39 kg CO ₂ €	Consumption of $\geq$ of meat per day. ⁷
Medium-Meat Diet	5.63 kg CO ₂ €	1.87 kg CO ₂ €	Consumption of 50-99g of meat per day. ⁷
Low-Meat Diet	4.67 kg CO ₂ €	1.55 kg CO ₂ €	Consumption of $<$ of meat per day. ⁷
Fish-Eater (Pescatarian)	3.91 kg CO ₂ €	1.30 kg CO ₂ €	Avoids terrestrial meat; consumes

			fish and dairy. ⁷
Vegetarian Diet	3.81 kg CO ₂ €	1.27 kg CO ₂ €	Consumes dairy and eggs; avoids all animal flesh. ⁷
Vegan Diet	2.89 kg CO ₂ €	0.96 kg CO ₂ €	Strictly plant-based; avoids all animal derivatives. ⁷

The conversion formulas integrated into the application's logic layer are expressed as:

$$E_{diet_daily} = EF_{diet_type}$$

$$E_{meal} = \frac{EF_{diet_type}}{3}$$

Developer Implementation: The Calculation Module

To strictly maintain the sub-10 megabyte repository limit, the inclusion of massive external libraries for data handling is strictly prohibited. The emission factors detailed above must be codified into an immutable TypeScript constant module. This architectural decision ensures that the calculation engine is heavily optimized, globally accessible within the application state, and statically typed to prevent runtime calculation errors. By utilizing TypeScript's as const assertion, the compiler ensures that these factors remain read-only, preventing accidental mutation during state changes.

```
TypeScript
// utils/calculationEngine.ts

/**
 * Universal Carbon Emission Factors Module
 * Sourced from DEFRA 2024/2025, EPA, and Scarborough et al. (2014)
 * All values expressed in kg CO2e.
 */
```

```

export const TRANSPORT_FACTORS_KG_KM = {
  CAR_GASOLINE 0.21 // kg CO2e per km (Includes WTT)
  CAR_DIESEL 0.21 // kg CO2e per km (Includes WTT)
  CAR_EV_AVERAGE 0.06 // kg CO2e per km (Grid average dependent)
  BUS_TRANSIT 0.13 // kg CO2e per passenger-km
  SUBWAY 0.02 // kg CO2e per passenger-km
} as const

```

```

export const HOME_ENERGY_FACTORS = {
  NATURAL_GAS_M3 2.50 // kg CO2e per cubic meter
  NATURAL_GAS_KWH 0.18290 // kg CO2e per kWh [13]
} as const

```

```

export const DIETARY_FACTORS_KG_DAY = {
  HIGH_MEAT 7.19 // kg CO2e per day
  MEDIUM_MEAT 5.63 // kg CO2e per day
  LOW_MEAT 4.67 // kg CO2e per day
  PESCATARIAN 3.91 // kg CO2e per day
  VEGETARIAN 3.81 // kg CO2e per day
  VEGAN 2.89 // kg CO2e per day
} as const

```

```

export type TransportMode = keyof typeof TRANSPORT_FACTORS_KG_KM
export type DietType = keyof typeof DIETARY_FACTORS_KG_DAY

```

```

/**
 * Calculates deterministic transport emissions based on mode and distance.
 */
export const calculateTransportEmissions = (
  mode: TransportMode,
  distanceKm: number,
  number => number
) => {
  return distanceKm * TRANSPORT_FACTORS_KG_KM[mode]
}

```

```

/**
 * Calculates dietary footprint based on selected diet type and meal quantity.
 */
export const calculateMealEmissions = (
  dietType: DietType,
  meals: number = 1,
  number => number
) => {

```

```
return DIETARY_FACTORS_KG_DA * 3 * meals
```

2. External Data Integrations (APIs)

The fundamental constraint of a ten-megabyte repository prohibits the local storage of the extensive, high-resolution datasets required for precise carbon calculation. Global geocoding databases, highly detailed road network topologies for distance routing, and historical time-series datasets of global electricity grid carbon intensities routinely span gigabytes of storage. Therefore, the system architecture mandates a decentralized approach, relying entirely on real-time data ingestion via external, developer-friendly edge APIs. This methodology not only preserves the lightweight nature of the source tree but also ensures that the application continuously leverages the most up-to-date environmental metrics without requiring client-side software updates.

Real-Time Grid Carbon Intensity API

To accurately determine the dynamic carbon impact of home energy usage or the lifecycle emissions of charging a Battery Electric Vehicle, the application requires access to localized grid intensity data. The carbon intensity of an electrical grid fluctuates continually based on demand surges, weather conditions affecting renewable generation, and the operational status of fossil fuel baseload plants.²¹

The primary recommended integration for this purpose is the **Emissions.dev API**. This service provides a highly optimized, low-latency REST endpoint (averaging sub-100ms response times globally) that calculates Scope 2 electricity emissions for over one hundred countries and all fifty United States.²² Crucially, it allows developers to pass raw energy consumption values

directly into the request, returning the computed greenhouse gas protocol-compliant CO_2e values, thereby offloading complex mathematical operations from the client device.

- **Service:** Emissions.dev Electricity API
- **API Endpoint URL:** <https://api.emissions.dev/v1/electricity/emissions>²³
- **HTTP Method:** GET
- **Authentication:** Requires a Bearer token passed explicitly within the HTTP Authorization header (e.g., Authorization: Bearer YOUR_API_KEY).²³
- **Query Parameters:**
 - kwh (Required, Number): Total electricity consumed in kilowatt-hours.²⁴
 - country (Optional, String): ISO 3166-1 alpha-2 country code designation.²⁴
 - state (Optional, String): State code for enhanced granularity in applicable regions.²⁴

Sample JSON Response Payload (Emissions.dev)

The response from the Emissions.dev API is structured hierarchically, separating the calculated empirical data from the methodological metadata. This structure is highly advantageous for parsing within a lightweight client framework.


```

JSON
{
  "data": {
    "type": "electricity_emission",
    "id": "a1b2c3d4-e5f6-7890",
    "attributes": {
      "emissions": {
        "co2e": 14.7,
        "co2e_unit": "kg",
        "co2e_calculation_method": "location_based_method",
        "source_trail": {}
      },
      "parameters": {
        "kwh": 42,
        "country": "US",
        "state": "FL"
      }
    },
    "meta": {
      "methodology": "GHG Protocol Scope 2 Guidance",
      "emission_factors_year": 2025,
      "calculated_at": "2026-06-08T12:00:00Z"
    }
  }
}

```

Alternative Contingency: Should the primary API experience downtime, the **Carbon Interface API** serves as a robust secondary integration. Utilizing a similar REST architecture, the Carbon Interface Estimates endpoint (<https://www.carboninterface.com/api/v1/estimates>) accepts a POST request containing a JSON payload detailing the activity.²⁵ A vehicle estimate request, for example, requires the submission of a specific `vehicle_model_id` alongside distance metrics, which subsequently generates a precise carbon estimate based on the make and model of the car.²⁵

Public Transit and Driving Distance Routing API

Estimating the carbon footprint of daily commuting or long-distance travel requires highly accurate distance calculations. Native HTML5 Geolocation APIs can calculate the linear, point-to-point distance between two coordinates using the haversine formula. However, actual

routing distances traversed by automobiles or public transit networks deviate significantly from linear paths due to road network topologies, geographical barriers, and urban infrastructure. Commercial mapping solutions like Google Maps or Mapbox provide excellent routing but require the installation of heavy client-side software development kits (SDKs) that violate the repository constraints.

The solution is the **OpenRouteService (ORS) API**. Developed by the Heidelberg Institute for Geoinformation Technology, ORS provides an open-source, highly accurate routing matrix and directions service based on OpenStreetMap data.²⁷ By calling the ORS directions endpoint, the application can determine the precise distance of a journey without rendering a visual map, thereby saving massive amounts of client-side memory.

- **Service:** OpenRouteService Directions API
- **API Endpoint URL:** <https://api.openrouteservice.org/v2/directions/{profile}>²⁸
 - The {profile} path variable dictates the routing algorithm and can be defined as driving-car, cycling-regular, or foot-walking.²⁹
- **HTTP Method:** POST (A POST method is heavily preferred over GET to bypass URL length restrictions when transmitting complex coordinate arrays).²⁹
- **Authentication:** Requires an API Key passed within the HTTP Authorization header.²⁸

Sample JSON Request Payload (OpenRouteService)

The payload requires an array of coordinate tuples, specifically formatted as [longitude, latitude].

```
JSON
{
  "coordinates": [8.681495, 49.41461]
}
```

Sample JSON Response Payload (OpenRouteService)

The ORS API returns a comprehensive GeoJSON object detailing the route geometry, segment steps, and turn-by-turn instructions.²⁷ For the purpose of carbon tracking, the vast majority of this data is superfluous.

```

JSON
{
  "type": "FeatureCollection",
  "bbox": [8.681495, 49.41461, 8.687872, 49.420318],
  "features": [
    {
      "type": "Feature",
      "properties": {
        "segments": [
          {
            "distance": 1204.5,
            "duration": 185.3,
            "steps": [
              [
                [
                  "summary",
                  {
                    "distance": 1204.5,
                    "duration": 185.3,
                    "geometry": {
                      "coordinates": [
                        "type": "LineString",
                        [
                          [
                            [
                              "metadata"
                            ]
                          ]
                        ]
                      ]
                    }
                  ]
                ]
              ]
            ]
          }
        ]
      }
    }
  ]
}

```

Architectural Implementation Note: To ensure the client application remains ultra-lightweight and performant, the frontend logic must intercept this response and immediately extract solely the features.properties.summary.distance value. Because this value is returned in meters, it must be programmatically divided by one thousand to derive the kilometer metric required by the calculation engine. Crucially, the remainder of the GeoJSON object—including the massive array of coordinate geometries—must be immediately discarded to trigger garbage collection and prevent memory leakage within the browser context.

3. The Insight Matrix (Behavioral Psychology)

Passive data tracking and visualization are fundamentally insufficient to induce sustained behavioral modification in users. Without active intervention, applications quickly succumb to user fatigue. To transform the platform from a mere reporting tool into a catalyst for environmental impact, the application must deploy a highly targeted behavioral intervention framework.

This architecture utilizes principles derived from **Nudge Theory**, pioneered by behavioral economists Richard Thaler and Cass Sunstein, which posits that positive reinforcement and indirect suggestions can influence decision-making more effectively than forced compliance.

Furthermore, the implementation relies heavily on the **Fogg Behavior Model** ($B = MAP$), which states that Behavior (B) only occurs when Motivation (M), Ability (A), and a Prompt (P) converge simultaneously. The application evaluates the user's continuous input telemetry stream in real-time. When specific, mathematically proven conditional thresholds are met, the system triggers contextual nudges—the Prompts—at the exact moment when the user has the Ability to make a choice.

To maximize user engagement and cognitive retention, the system relies on two critical psychological heuristics:

1. **Salience:** Presenting the specific environmental information that is most relevant at the exact moment of decision-making, cutting through cognitive noise.
2. **Gamification and Loss Aversion:** Translating abstract, invisible carbon savings into immediate, quantifiable achievements, while framing detrimental actions as tangible losses.

The Micro-Actions Matrix

The following structured matrix outlines ten personalized micro-actions. Each action is bound to a specific telemetry trigger derived from the user's input state. The estimated carbon dioxide savings are calculated explicitly utilizing the exact coefficients and empirical data established in Section 1.⁷

Micro-Action Directive	Psychological Mechanism	Estimated Carbon Savings	Conditional Trigger (Logic Map)
1. The "Meatless Monday" Nudge	Gamification / Commitment Consistency	4.30 kg CO ₂ € per day. <i>(Derived by subtracting the Vegan factor [2.89] from the High Meat</i>	IF user logs Diet = "HIGH_MEAT" for > consecutive days, TRIGGER Nudge: "Swap tomorrow's meals to plant-based to

		<i>factor [7.19]).⁷</i>	<i>completely offset the carbon from your last commute."</i>
2. The Subway Substitution	Saliency / Choice Architecture	1.90 kg CO₂€ per 10km urban trip. <i>(Calculated as 10km ×).¹⁰</i>	IF user plans route with Transport = "CAR" AND Distance < 15km AND Location = "Urban Center", TRIGGER Nudge: "A subway trip today avoids enough carbon to charge your smartphone every day for a year."
3. EV Off-Peak Grid Charging	Loss Aversion / Financial Incentive	Variable (~ 0.5 —) depending entirely on the API intensity delta.²¹	IF Transport = "CAR_EV" AND Emissions.dev API current_intensity > 24hr_average, TRIGGER Nudge: "The local power grid is currently burning fossil fuels to meet peak demand. Delay EV charging until 11 PM to cut charging emissions by 30%."
4. Thermostat Calibration Drop	Anchoring Effect	~ per day. <i>(Based on a standard 10% efficiency gain reducing average</i>	IF Season = "Winter" AND Home_Energy_Gas > Average_Threshold, TRIGGER Nudge: "Lowering your thermostat by just

		3 m³ daily winter gas usage).¹¹	1°C reduces your heating footprint by 10% without affecting ambient comfort."
5. Active Transport Advocacy	Endowed Progress Effect	2.10 kg CO₂€ per 10km trip. (Calculated as 10km × Gasoline Car 0.21).¹⁰	IF user logs Transport = "CAR" AND Distance < 5km AND Weather API = "Clear", TRIGGER Nudge: "Short combustion engine trips emit the highest carbon per km. Cycling this route avoids 2.1 kg of CO₂e."
6. The Flexitarian Shift	Social Proof / Herd Behavior	1.56 kg CO₂€ per day. (Calculated by subtracting Medium Meat [5.63] from High Meat [7.19]).⁷	IF user logs Diet = "HIGH_MEAT", TRIGGER Nudge: "70% of active users in your community tier have switched one beef meal to poultry this week, saving 1.5 kg CO₂e."
7. Bus Transit Optimization	Default Bias	0.80 kg CO₂€ per 10km trip. (Calculated as 10km ×).¹⁰	IF user requests distance calculation between two points on known municipal transit corridors, TRIGGER Nudge: "Taking the bus for this specific route cuts your transit footprint by nearly 40%."

8. High-Efficiency Boiling	Micro-Habit Stacking	0.03 kg CO ₂ € per boil action.	IF user interacts with the application during morning hours (06:00 - 09:00), TRIGGER Nudge: <i>"Only boil the exact volume of water you need for your morning coffee. It prevents 30g of wasted CO₂e."</i>
9. Cold Water Laundry Framing	Framing Effect	0.40 kg CO ₂ € per laundry load.	IF Day = "Weekend" (statistically common chore period), TRIGGER Nudge: <i>"Washing textiles on a cold cycle reduces washing machine operational carbon emissions by up to 90%."</i>
10. Digital Fasting Prompt	Curiosity Gap	0.10 kg CO ₂ € per hour of HD video streaming.	IF local Time > 22:00, TRIGGER Nudge: <i>"High-definition streaming relies on energy-intensive data centers. Read a book tonight to bank 100 grams of CO₂e."</i>

Algorithmic Implementation of the Insight Matrix

Within the application's global state management architecture, the logic map operates as an asynchronous event listener evaluating the array of recent user inputs. To adhere to the strict architectural constraints, this logic is executed entirely via a minimal, client-side JavaScript rule engine, requiring zero external decision-engine dependencies.

JavaScript

```
/**
 * Lightweight Behavioral Insight Engine
 * Evaluates user telemetry against the psychological logic map to yield targeted nudges.
 */
function evaluateBehavioralNudges(userState, externalApiData)
const generatedNudges =

// Trigger 1: The "Meatless Monday" Nudge (Commitment Consistency)
if userState.dietLog.consecutiveHighMeatDays > 4
  generatedNudges.push(
    id 'nudge_meatless_shift'
    message 'Swap tomorrow\'s meals to a plant-based diet to completely offset the carbon from
your last commute.'
    potentialSavingsKg 4.30
    psychologicalType 'GAMIFICATION'
  )

// Trigger 2: The Subway Substitution (Salience)
const mostRecentTrip = userState.transportLog
if mostRecentTrip && mostRecentTrip.mode === 'CAR_GASOLINE' &&
mostRecentTrip.distanceKm > 15
  const savings = (mostRecentTrip.distanceKm * 0.21 - (mostRecentTrip.distanceKm *
0.02
  generatedNudges.push(
    id 'nudge_subway_substitution'
    message `A subway trip today avoids ${savings.toFixed(2)} kg CO2e, enough to charge a
smartphone for a year.`
    potentialSavingsKg savings
    psychologicalType 'SALIENCE'
  )

// Trigger 3: EV Off-Peak Grid Charging (Loss Aversion)
if userState.ownsEV && externalApiData.currentGridIntensity >
externalApiData.averageGridIntensity)
```



```

generatedNudges.push({
  id: 'nudge_ev_charging',
  message: 'The local power grid is burning fossil fuels to meet peak demand. Delay EV charging
until 11 PM to cut charging emissions.',
  potentialSavingsKg: null // Dynamic based on API delta
  psychologicalType: 'LOSS_AVERSION'
})

return generatedNudges

```

4. Technical Architecture & UI Guidelines

Constructing a visually striking, data-intensive web application that operates strictly within a sub-10 megabyte repository requires extreme dependency austerity. The traditional MERN stack (MongoDB, Express, React, Node.js) relies heavily on massive client-side React bundles, routing libraries, and state management tools that rapidly inflate the build size. Consequently, modern server-rendered frameworks present a superior architectural foundation for this specific engineering challenge.

Architecture Strategy: Next.js App Router Paradigm

The platform will be constructed utilizing the **Next.js App Router (/app directory)**. This architecture represents a paradigm shift by natively supporting React Server Components (RSC). By executing carbon factor parsing, complex mathematical arrays, API fetching requests to Emissions.dev and OpenRouteService, and the Insight Matrix calculations entirely on the server, the application strips out the vast majority of traditional JavaScript payload. The client browser receives highly optimized, serialized HTML alongside only the minimal hydration JavaScript necessary for interactive elements like buttons and forms. This fundamentally ensures the source tree and build output remain well below the strict file size limits.

Strict Dependency Optimization Strategy

To guarantee the repository adheres to the stringent size restrictions, developers must operate under the following architectural mandates:

1. **Zero UI Component Libraries:** The project strictly forbids the installation of comprehensive UI component libraries such as Material-UI, Ant Design, or Chakra UI. The requisite CSS-in-JS engines and pre-built component ecosystems inherent to these libraries bloat the node_modules directory and final payload unacceptably. All UI components will be constructed using raw, semantic HTML nodes styled exclusively with utility classes.
2. **Prohibition of Charting Libraries:** Data visualization libraries like D3.js, Chart.js, Recharts, or Highcharts are strictly prohibited. These dependencies carry massive payload sizes and complex rendering engines. Instead, the application will render

required data visualizations using **CSS-only graphics and dynamically calculated inline Scalable Vector Graphics (SVGs)**.

3. **Elimination of Date Parsing Libraries:** Heavy temporal libraries such as Moment.js or Date-fns will not be utilized. The native JavaScript Intl.DateTimeFormat API and standard Date objects possess all requisite capabilities for tracking the temporality of user carbon logs.
4. **Native Network Requests:** External network requests to the APIs mapped in Section 2 will rely entirely on the native Node.js fetch API, completely eliminating the need for libraries such as Axios.

Data Visualization: The Native SVG Trigonometric Approach

Rather than importing a megabyte-heavy library to render a simple dashboard chart, a comprehensive doughnut chart—representing the user's categorized carbon footprint breakdown across Transport, Home Energy, and Diet—can be rendered natively. By mathematically calculating the SVG stroke-dasharray dynamically based on the percentage values of the user data, developers can create responsive, animated charts utilizing zero kilobytes of external dependencies.

```
JavaScript
// components/NativeSvgDonutChart.tsx
import React from 'react'

interface ChartDataPoint {
  label: string
  value: number
  hexColor: string
}

/**
 * Renders a lightweight, dependency-free SVG Donut Chart.
 * Utilizes the stroke-dasharray technique to calculate slice circumferences.
 */
export default function NativeSvgDonutChart({ data }: { data: ChartDataPoint }) {
  const totalValue = data.reduce((sum, item) => sum + item.value, 0)
  let cumulativePercentage = 0

  return (
    <svg viewBox="0 0 32 32" className="w-full h-full transform -rotate-90 drop-shadow-md">
```

```

    {data.map((slice, index) => {
      const percentage = (slice.value / totalValue) * 100;
      // The circumference of a circle with r=16 is approx 100.53,
      // utilizing viewBox 32x32 scales the stroke to percentage effectively.
      const dashArray = `${percentage} ${100 - percentage}`;
      const dashOffset = -cumulativePercentage;

      cumulativePercentage += percentage;

      return (
        <circle
          key={index}
          r="16"
          cx="16"
          cy="16"
          fill="transparent"
          stroke={slice.hexColor}
          strokeWidth="4"
          strokeDasharray={dashArray}
          strokeDashoffset={dashOffset}
          className="transition-all duration-1000 ease-out"
          aria-label={` ${slice.label}: ${slice.value} kg CO2e`}
        />
      );
    })}
  </svg>

```

UI Aesthetic: The Cyber-Themed Dark Mode

The visual aesthetic directive requires a cyber-themed, dark mode interface utilizing neon cyan and purple accents. This specific visual language is highly strategic. It seamlessly synergizes with the concept of environmental tracking by presenting sustainability data through a futuristic, high-technology lens. This subconsciously communicates to the user that sustainability is not a regressive lifestyle, but rather a modern, data-driven optimization of daily living. Furthermore, dark mode interfaces physically consume less power on modern OLED smartphone displays, generating a literal, albeit microscopic, energy-saving mechanism that ties directly into the application's core environmental mission.

Tailwind CSS Configuration Strategy

Tailwind CSS will be deployed as the exclusive styling mechanism. Because the Tailwind compiler utilizes strict tree-shaking algorithms to purge unused CSS classes during the production build process, the final CSS payload is typically reduced to less than 10 KB. This aligns perfectly with the repository size constraints.

The tailwind.config.js file is the sole mechanism for establishing the exact color palette and the specific glow effects (simulated via drop shadows) required for the neon cyber aesthetic, explicitly removing any reliance on external graphic image assets.

```
JavaScript
// tailwind.config.js
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: [
    './app/**/*.{js,ts,jsx,tsx}',
    './components/**/*.{js,ts,jsx,tsx}',
  ],
  theme: {
    extend: {
      colors: {
        cyber: {
          dark: '#0a0a0f' // Deep void background for OLED efficiency
          panel: '#13131a' // Elevated interface panel background
          cyan: '#00f3ff' // Primary neon data accent
          purple: '#b026ff' // Secondary neon accent for gamification elements
          alert: '#ff0055' // Warning/Error state for high emissions
          muted: '#4a4a5e' // Muted text for secondary telemetry
        },
      },
      boxShadow: {
        'neon-cyan': '0 0 10px #00f3ff, 0 0 20px #00f3ff',
        'neon-purple': '0 0 10px #b026ff, 0 0 20px #b026ff',
      },
      fontFamily: {
        mono: '"Fira Code"' 'monospace' // Typeface for empirical data displays
        sans: '"Inter"' 'sans-serif' // Primary UI typeface
      },
    },
  },
  plugins: [],
}
```

Content Delivery Network (CDN) Asset Optimization

To further eliminate repository bloat and ensure the source tree remains purely focused on logical code, all requisite fonts, iconography, and static graphical assets must bypass the local repository structure entirely.

1. **Typography via Edge Delivery:** Fonts such as *Inter* (necessary for standard UI legibility) and *Fira Code* (utilized to reinforce the cyber-aesthetic within empirical data displays) must be loaded dynamically via Google Fonts utilizing the Next.js next/font module, which optimizes these requests at the edge during the build process without storing font files locally.
2. **Iconography Strategy:** Instead of executing an NPM installation for exhaustive icon sets such as react-icons or FontAwesome, which inevitably introduce thousands of unused SVG files into the dependency tree, the application will utilize raw SVG strings embedded directly within the React components. Should dynamic iconography be strictly required, lightweight unpkg or jsDelivr CDN links will serve the individual SVG vectors upon request.
3. **UI Component Interaction Design:** The interface will utilize stark, high-contrast, one-pixel borders to define hierarchy. Interactive elements, such as data input fields and submission buttons, will feature baseline border-cyber-cyan styling with an applied hover:shadow-neon-cyan utility class. This CSS-only methodology creates immediate, highly satisfying tactile feedback for the user without requiring a single kilobyte of JavaScript animation overhead.

By strictly adhering to these comprehensive architectural paradigms—utilizing server-side rendering logic to mitigate client payloads, executing direct deterministic mathematical calculations with peer-reviewed scientific constants, integrating lightweight edge-routing and carbon API endpoints, and rendering all user interface components via native SVG mathematics and CDN-delivered assets—the resulting platform will comfortably operate well below the stringent ten-megabyte repository limit. Simultaneously, it will provide an exhaustively detailed, highly scalable, and psychologically engaging carbon footprint tracking experience that empowers users to actively mitigate their environmental impact.

Works cited

1. UK Government GHG Conversion Factors for Company Reporting - GOV.UK, accessed on June 8, 2026, <https://assets.publishing.service.gov.uk/media/6846a4f55e92539572806125/ghg-conversion-factors-2025-full-set.xlsx>
2. Greenhouse gas reporting: conversion factors 2025 - GOV.UK, accessed on June 8, 2026, <https://www.gov.uk/government/publications/greenhouse-gas-reporting-conversion-factors-2025>
3. Greenhouse gas reporting: conversion factors 2024 - GOV.UK, accessed on June 8, 2026, <https://www.gov.uk/government/publications/greenhouse-gas-reporting-conversion-factors-2024>

[on-factors-2024](#)

4. DEFRA GHG Emission Conversion Factors Updates 2024 - MyCarbon, accessed on June 8, 2026,
<https://www.mycarbon.co.uk/2024/09/06/defra-ghg-emission-conversion-factors-update-2024/>
5. GHG Emission Factors Hub | US EPA, accessed on June 8, 2026,
<https://www.epa.gov/climateleadership/ghg-emission-factors-hub>
6. Emission Factors for Greenhouse Gas Inventories - EPA, accessed on June 8, 2026,
<https://www.epa.gov/system/files/documents/2024-02/ghg-emission-factors-hub-2024.pdf>
7. Dietary greenhouse gas emissions of meat-eaters, fish-eaters, vegetarians and vegans in the UK - PMC, accessed on June 8, 2026,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC4372775/>
8. Carbon and Water Footprints of Diet Choices | Animal Charity Evaluators, accessed on June 8, 2026,
<https://animalcharityevaluators.org/wp-content/uploads/2022/04/carbon-and-water-footprints-of-diet-choices.pdf>
9. UNDERSTANDING GREENHOUSE GAS EMISSIONS FROM NATURAL GAS — EPA 2023 INVENTORY (1990–2021) - American Gas Association, accessed on June 8, 2026,
https://www.aga.org/wp-content/uploads/2023/10/AGA-Report_Understanding-GHG-Emissions-from-Natural-Gas_2023.pdf
10. greenhouse gas report for 2024 - SME Climate Hub, accessed on June 8, 2026,
<https://smeclimatehub.org/wp-content/uploads/2023/04/MINI-A-TURE-2024-GHG-report.pdf>
11. 2024 Government greenhouse gas conversion factors for company reporting: Methodology paper - GOV.UK, accessed on June 8, 2026,
<https://assets.publishing.service.gov.uk/media/66a9fe4ca3c2a28abb50da4a/2024-greenhouse-gas-conversion-factors-methodology.pdf>
12. Greenhouse gas emission calculator, accessed on June 8, 2026,
https://www.financelatvia.eu/wp-content/uploads/2025/04/GHG_Calculator_guidelines.pdf
13. C23042 EMR Conversion factors - HESA, accessed on June 8, 2026,
https://www.hesa.ac.uk/collection/c23042/download/conversion_factors.pdf
14. Conversion factors CO2 emissions - Sustainability Report 2024, accessed on June 8, 2026,
<https://sustainabilityreport.zeppelin.com/2024/appendix/conversion-factors-co2-emissions/>
15. Travel API | emissions.dev Documentation, accessed on June 8, 2026,
<https://emissions.dev/docs/api/travel>
16. India Electricity Carbon Footprint & Environmental Impact - Consumer Ecology, accessed on June 8, 2026,
<https://consumerecology.com/india-electricity-carbon-footprint-environmental-impact/>

17. Carbon emissions in tCO₂ at IIT (BHU) Varanasi, accessed on June 8, 2026, https://iitbhu.ac.in/contents/institute/admin/doc/admin_carbon_emissions%20in_co2.pdf
18. CO₂ Baseline Database for the Indian Power Sector User Guide - Central Electricity Authority, accessed on June 8, 2026, https://cea.nic.in/wp-content/uploads/baseline/2025/12/User_Guide_V_21.0.pdf
19. Why eat veggie? - Vegetarian Society, accessed on June 8, 2026, <https://vegsoc.org/eating-veggie/why-eat-veggie/>
20. (PDF) Dietary greenhouse gas emissions of meat-eaters, fish-eaters, vegetarians and vegans in the UK - ResearchGate, accessed on June 8, 2026, https://www.researchgate.net/publication/263353807_Dietary_greenhouse_gas_emissions_of_meat-eaters_fish-eaters_vegetarians_and_vegans_in_the_UK
21. Carbon Intensity API, accessed on June 8, 2026, <https://carbonintensity.org.uk/>
22. emissions.dev: Free Carbon Emissions API for Developers, accessed on June 8, 2026, <https://emissions.dev/>
23. API Reference | emissions.dev Documentation, accessed on June 8, 2026, <https://emissions.dev/docs/api>
24. Electricity API | emissions.dev Documentation, accessed on June 8, 2026, <https://emissions.dev/docs/api/electricity>
25. Carbon Interface | Docs, accessed on June 8, 2026, <https://docs.carboninterface.com/>
26. Electricity estimate tutorial - Carbon Interface, accessed on June 8, 2026, <https://www.carboninterface.com/tutorials/electricity>
27. Matrix Endpoint | openrouteservice backend documentation - GitHub Pages, accessed on June 8, 2026, <https://giscience.github.io/openrouteservice/api-reference/endpoints/matrix/>
28. API interactive examples | ORS - openrouteservice, accessed on June 8, 2026, <https://openrouteservice.org/dev/>
29. openrouteservice-docs/API V2/swagger.json at master - GitHub, accessed on June 8, 2026, <https://github.com/GIScience/openrouteservice-docs/blob/master/API%20V2/swagger.json>